

So sánh các Mô hình trong Hệ thống

Secure Virtual Assistant with Speaker Recognition
Mỗi model làm nhiệm vụ gì — và khi nào nên chọn

Đề án: Secure-VA

Tháng 6, 2026

Abstract

Trợ lý ảo gồm một **pipeline xử lý giọng nói** với bốn khối, mỗi khối có thể hoán đổi mô hình qua file `.env` mà không sửa code. Tài liệu này so sánh các lựa chọn ở từng khối: **(1) ASR** (giọng nói → văn bản), **(2) Speaker Encoder** (giọng nói → embedding để nhận diện/xác minh), **(3) NLU** (văn bản → ý định), **(4) TTS** (văn bản → giọng nói). Với mỗi mô hình: nêu *nhiệm vụ*, *ưu/nhược điểm*, và *khi nào nên chọn*.

Contents

1	Toàn cảnh pipeline	2
2	Khối 1 — ASR: Nhận dạng tiếng nói	2
3	Khối 2 — Speaker Encoder: Sinh trắc giọng nói	3
3.1	RawNet3 và vì sao đề án chỉ tích hợp 3 lựa chọn	4
4	Khối 3 — NLU: Hiểu ý định	4
5	Khối 4 — TTS: Tổng hợp giọng nói	5
6	Tóm tắt: bảng quyết định nhanh	6
7	Muốn đổi hoặc thêm model thì sửa ở đâu?	6
7.1	Mức 1 — Đổi sang model ĐÃ hỗ trợ (chỉ sửa <code>.env</code>)	6
7.2	Mức 2 — THÊM một model mới (vd RawNet3)	6

1 Toàn cảnh pipeline

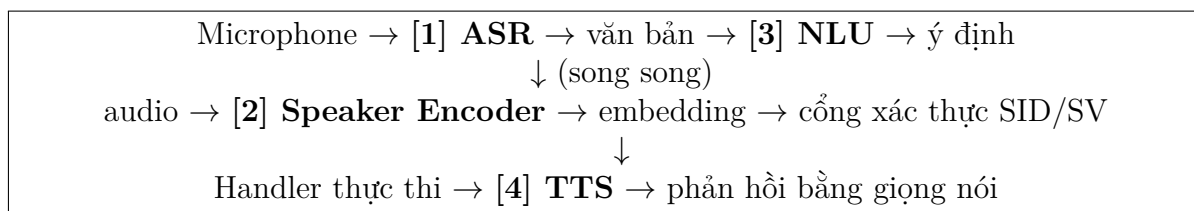


Table 1: Cấu hình **đang dùng** của đồ án (đọc từ `.env`).

Khối	Model đang bật	Biến <code>.env</code>
ASR	PhoWhisper-base	ASR_BACKEND=phowhisper
Speaker Encoder	ECAPA-TDNN (checkpoint VIVOS)	SPEAKER_BACKEND=ecapa
NLU	Gemini 2.5-flash (fallback rule-based)	GEMINI_MODEL=gemini-2.5-flash
TTS	gTTS (fallback pyttsx3)	—

2 Khối 1 — ASR: Nhận dạng tiếng nói

Nhiệm vụ: Chuyển audio giọng nói thành văn bản (speech-to-text) để NLU hiểu lệnh. Cài đặt trong `core/asr.py`, chọn backend qua `ASR_BACKEND`.

Table 2: So sánh hai backend ASR.

	faster-whisper	PhoWhisper (đang dùng)
Bản chất	Whisper (OpenAI) đa ngôn ngữ, nén CTranslate2 int8	Whisper fine-tune 844 giờ tiếng Việt (VinAI)
Độ chính xác (tiếng Việt)	Khá; dễ "English-hoá" câu ngắn	Cao hơn rõ ở cùng kích thước model
Tốc độ	Nhanh trên CPU (int8)	Chậm hơn (transformers eager mode)
Đa ngôn ngữ	Có (nhiều thứ tiếng)	Chủ yếu tiếng Việt
Kích thước	tiny / base / small / medium / large-v3	base / small / medium / large
Phụ thuộc	faster-whisper	transformers (+ GPU nếu lớn)

Khi nào chọn cái nào?

- **Chọn PhoWhisper** khi: ứng dụng *chỉ phục vụ tiếng Việt* và ưu tiên độ chính xác — đúng trường hợp đồ án này. Dùng `base` trên CPU, `small/medium` nếu có GPU.
- **Chọn faster-whisper** khi: cần *tốc độ trên CPU yếu*, cần *đa ngôn ngữ*, hoặc làm demo nhanh. Trên CPU nên đặt `WHISPER_MODEL=base`; có GPU thì `medium/large-v3` + `WHISPER_COMPUTE=float16`.

Lưu ý chung: cả hai đều được tăng cường bằng *initial prompt bias* (ưu tiên từ-khoá-lệnh) và *bộ lọc hallucination* (loại câu "outro YouTube" khi audio quá ngắn/im lặng).

3 Khối 2 — Speaker Encoder: Sinh trắc giọng nói

Nhiệm vụ: Biến đoạn giọng nói thành một *vector embedding* đặc trưng cho người nói. So embedding bằng **cosine similarity** để: nhận diện người nói (**SID**) và xác minh danh tính (**SV**). Cài đặt trong `core/speaker_encoder.py`, chọn qua `SPEAKER_BACKEND`.

Đây là *lỗi bảo mật* của hệ thống — quyết định ai được chạy tác vụ nhạy cảm.

Table 3: So sánh ba lựa chọn speaker encoder.

	ECAPA (VIVOS) <i>đang dùng</i>	tự train	ECAPA pretrained (SpeechBrain)	WavLM-SV (Microsoft)
Kiến trúc	ECAPA-TDNN, embed- ding 192-d		ECAPA-TDNN, 192-d	WavLM + X-vector head, 512-d
Huấn luyện trên	VIVOS (65 người, tiếng Việt)		VoxCeleb2 (general)	VoxCeleb1 verification
Hiệu năng	SV EER 0.96% (đo nội bộ)		Tổng quát, chưa tối ưu tiếng Việt	EER thấp trên VoxCeleb1-O; bắt impostor giọng giống tốt
Ngưỡng SV / SID	0.45 / 0.35		0.25 / 0.20	0.93 / 0.90
Phụ thuộc	SpeechBrain + checkpoint		SpeechBrain (tải tự động)	transformers (nặng hơn)
Kích hoạt	Khi có file checkpoint hợp lệ		Tự fallback khi thiếu checkpoint	wavlm

Vì sao ngưỡng khác nhau? Mỗi model có *phân phối điểm cosine* khác nhau, nên ngưỡng chấp nhận phải hiệu chỉnh riêng cho từng backend (vd WavLM điểm tập trung rất cao → ngưỡng 0.93). **Không** dùng chung một ngưỡng.

Khi nào chọn cái nào?

- **ECAPA tự train (VIVOS)** — *mặc định cho đồ án*: khớp miền tiếng Việt, EER thấp nhất trong các bằng chứng nội bộ. Chọn khi đã có checkpoint và phục vụ người dùng Việt.
- **ECAPA pretrained**: chọn khi *chưa kịp train* hoặc cần baseline chạy ngay — hệ thống tự dùng khi không tìm thấy checkpoint, không cần cấu hình.
- **WavLM-SV**: chọn khi cần *độ phân biệt impostor cao* (chống giọng giống nhau) và chấp nhận model nặng hơn, tốt nhất có GPU. Phù hợp khi nâng cấp bảo mật hoặc so sánh học thuật.

Lưu ý chung: mọi backend dùng chung các kỹ thuật ổn định hoá — *multi-window averaging*, *AS-Norm-lite* (chuẩn hoá điểm theo cohort), và *lọc chất lượng centroid* khi đăng ký giọng.

3.1 RawNet3 và vì sao đề án chỉ tích hợp 3 lựa chọn

RawNet3 (Jung et al., 2022) là một mô hình speaker verification SOTA khác. Điểm khác biệt cốt lõi so với ECAPA: nó nhận *trực tiếp raw waveform*, **không** dùng mel-filterbank thủ công — thay vào đó học một front-end lọc tham số (kiểu sinc), rồi qua backbone Res2Net + attentive statistics pooling.

Table 4: RawNet3 so với hướng tiếp cận của đề án.

	RawNet3	ECAPA-TDNN (đề án dùng)
Đầu vào	Raw waveform (front-end học được)	Mel-filterbank 80-d + log + CMN
Backbone	Res2Net + attentive stat pooling	SE-Res2Block + attentive stat pooling
Tham số	~16M	~6.2M
Hiệu năng	EER $\approx 0.9\%$ trên VoxCeleb1-O (công bố gốc)	EER 0.96% trên VIVOS (đo nội bộ)
Hệ sinh thái	clovaaai/voxceleb_trainer, asv-subtools	SpeechBrain (pretrained + train sẵn)

Vì sao KHÔNG dùng RawNet3 trong đề án

RawNet3 *không yếu* — ngược lại, nó rất mạnh. Lý do không chọn là về **phạm vi và chi phí tích hợp**, không phải chất lượng:

1. **Kiến trúc chuẩn của YC1 là ECAPA-TDNN**: toàn bộ pipeline huấn luyện (FbankExtractor, train_ecapa.py, các checkpoint VIVOS/ VoxCeleb) được xây quanh mel + ECAPA. Đổi sang RawNet3 là làm lại YC1 từ đầu.
2. **Khác hệ sinh thái**: ECAPA và WavLM có sẵn API gọn trong SpeechBrain/transformers; RawNet3 nằm ở repo voxceleb_trainer, phải tự viết encoder class mới + thêm dependency + tải/convert weight.
3. **Phải hiệu chỉnh lại ngưỡng**: phân phối điểm cosine khác $\rightarrow$ phải calibrate SV_THRESHOLD/SID_MIN_THRESHOLD riêng từ đầu.
4. **Lợi ích biên nhỏ so với chi phí**: ECAPA-VIVOS đã đạt EER 0.96% trên miền tiếng Việt — đủ tốt cho demo. RawNet3 chủ yếu hơn ở benchmark VoxCeleb quy mô lớn; để phát huy trên tiếng Việt vẫn cần train lại.
5. **3 lựa chọn hiện tại đã phủ đủ tình huống**: own checkpoint (tốt nhất, đúng miền), pretrained (fallback chạy ngay), WavLM (đối chứng/độ phân biệt cao). RawNet3 sẽ là *lựa chọn thứ 4* hợp lý khi cần đẩy SOTA.

Quan trọng: kiến trúc backend đã *mở rộng được* — thêm RawNet3 chỉ cần viết một encoder class mới (xem Mục 7).

4 Khối 3 — NLU: Hiểu ý định

Nhiệm vụ: Từ văn bản, xác định *ý định* (intent) và *tham số* (entity), vd "gửi email cho Nam" $\rightarrow$ intent `send_email`, entity `recipient=Nam`. Ngoài ra trả lời câu hỏi tổng quát

(`general_question`). Cài đặt trong `core/nlu.py`, chọn theo việc có `GEMINI_API_KEY` hay không.

Table 5: So sánh hai cơ chế NLU.

	Gemini (function-calling) <i>đang dùng</i>	Rule-based (fallback)
Cách hoạt động	LLM hiểu câu tự nhiên, tự map sang hàm/intent + entity	Khớp từ khoá / mẫu (pattern) cố định
Độ linh hoạt	Cao — hiểu cách diễn đạt đa dạng, có ngữ cảnh	Thấp — chỉ nhận câu đúng mẫu
Trả lời tự do	Có (<code>general_question</code> qua Gemini)	Không
Chi phí	Cần API key + mạng + có quota	Miễn phí, offline, tức thời
Tính ổn định	Phụ thuộc dịch vụ (có thể 429/quota)	Deterministic

Khi nào chọn cái nào?

- **Gemini** — mặc định: khi cần hiểu lệnh nói tự nhiên, đa dạng và trả lời câu hỏi mở. Đề án dùng `gemini-2.5-flash`.
- **Rule-based** — tự động kích hoạt khi *thiếu API key, mất mạng hoặc hết quota*: đảm bảo các lệnh cốt lõi vẫn chạy. Cũng nên dùng khi cần chạy hoàn toàn offline/bảo mật dữ liệu.

Liên quan: Gemini cũng được dùng để *sửa lỗi transcript* ASR (bật qua `ASR_CORRECT`); mặc định chỉ sửa khi NLU không hiểu để tiết kiệm quota.

5 Khối 4 — TTS: Tổng hợp giọng nói

Nhiệm vụ: Chuyển văn bản phản hồi thành audio để đọc cho người dùng (text-to-speech). Cài đặt trong `core/tts.py`.

Table 6: So sánh hai engine TTS.

	gTTS (đang dùng)	pyttsx3 (fallback)
Bản chất	Google TTS online	Engine hệ điều hành (SAPI5/NSSpeech/espeak)
Chất lượng	Tự nhiên , nhiều ngôn ngữ (vi/en/ja/ko/zh/fr/de)	Giọng máy, kém tự nhiên hơn
Mạng	Cần Internet	Hoàn toàn offline

Khi nào chọn cái nào?

- **gTTS** — mặc định: chất lượng cao, đa ngôn ngữ theo `preferences.language` của người dùng.
- **pyttsx3** — *tự động* thay thế khi gTTS lỗi (mất mạng / rate limit), giữ cho demo không gián đoạn.

6 Tóm tắt: bảng quyết định nhanh

Table 7: Chọn model theo tình huống.

Tình huống	Lựa chọn khuyến nghị
Demo/đồ án tiếng Việt (mặc định)	PhoWhisper-base + ECAPA-VIVOS + Gemini + gTTS
Máy CPU yếu, cần nhanh	faster-whisper (base) + ECAPA + rule-based + gTTS
Chạy hoàn toàn offline / không mạng	faster-whisper + ECAPA + rule-based + pyttsx3
Chưa train được speaker model	ECAPA pretrained (tự fallback)
Cần chống giả mạo giọng mạnh hơn	WavLM-SV (có GPU)
Có GPU, tối đa độ chính xác tiếng Việt	PhoWhisper-medium + ECAPA-VIVOS + Gemini
Đẩy SOTA EER (nghiên cứu)	RawNet3 — <i>chưa tích hợp</i> , cần thêm encoder class (Mục 7)

7 Muốn đổi hoặc thêm model thì sửa ở đâu?

Hệ thống thiết kế theo *backend hoán đổi được*: mỗi khối đọc cấu hình từ `.env` qua `core/config.py`, rồi một hàm *factory* chọn class phù hợp. Có hai mức thay đổi.

7.1 Mức 1 — Đổi sang model ĐÃ hỗ trợ (chỉ sửa `.env`)

Không cần đụng code, chỉ đổi biến môi trường rồi khởi động lại app:

Khối	Biến trong <code>.env</code>	Giá trị hợp lệ
ASR	ASR_BACKEND	faster-whisper / phowhisper
	WHISPER_MODEL	tiny..large-v3
	PHOWHISPER_MODEL	vinai/PhoWhisper-base..large
Speaker	SPEAKER_BACKEND	ecapa / wavlm
	SPEAKER_CKPT	đường dẫn checkpoint ECAPA
	SV_THRESHOLD_*, SID_MIN_THRESHOLD_*	ngưỡng từng backend
NLU	GEMINI_MODEL GEMINI_API_KEY	/ model Gemini / bỏ trống = rule-based
TTS	TTS_LANG	vi/en/... (pyttsx3 tự fallback)

Table 8: Các điểm đổi model không cần sửa code.

7.2 Mức 2 — THÊM một model mới (vd RawNet3)

Cần sửa **3** nơi. Lấy ví dụ thêm backend speaker `rawnet3`:

(a) `core/config.py` — khai báo biến + ngưỡng:

```
SPEAKER_BACKEND = os.getenv("SPEAKER_BACKEND", "ecapa").lower()
RAWNET3_CKPT = os.getenv("RAWNET3_CKPT", "./checkpoints/rawnet3.pt")
SV_THRESHOLD_RAWNET3 = float(os.getenv("SV_THRESHOLD_RAWNET3", "0.30"))
SID_MIN_THRESHOLD_RAWNET3 = float(os.getenv("SID_MIN_THRESHOLD_RAWNET3", "0.25"))
```

(b) `core/speaker_encoder.py` — viết encoder class mới (chỉ cần đúng “hộp đồng”: có `encode()`, `embedding_dim`, `backend_id`, `sv_threshold`, `sid_min_threshold`):

```
class RawNet3SpeakerEncoder(SpeakerEncoder):
    embedding_dim = 256
    backend_id = "rawnet3"
    def __init__(self):
        self.device = "cuda" if torch.cuda.is_available() else "cpu"
        self._lock = threading.Lock()
        self.model = load_rawnet3(config.RAWNET3_CKPT).to(self.device).eval()
        self.mode = "rawnet3"
        self.sv_threshold = config.SV_THRESHOLD_RAWNET3 # ngưỡng riêng
        self.sid_min_threshold = config.SID_MIN_THRESHOLD_RAWNET3
    @torch.no_grad()
    def encode(self, audio): # raw waveform -> embedding L2-norm
        wav = torch.from_numpy(audio).float().unsqueeze(0).to(self.device)
        emb = self.model(wav) # RawNet3 nhận RAW WAVEFORM, không mel
        return F.normalize(emb, dim=-1).cpu().numpy()[0]
```

(c) `factory get_encoder()` — thêm một nhánh:

```
elif backend == "rawnet3":
    _encoder_instance = RawNet3SpeakerEncoder()
```

Vì sao thêm dễ như vậy?

Toàn bộ pipeline (router, SID/SV, enroll) chỉ gọi `encode(audio)` và đọc `embedding_dim/ngưỡng` — *không* phụ thuộc class cụ thể. Nhờ *factory pattern* + *Protocol* (xem `get_encoder()` và `get_asr()`), thêm model mới không phải sửa router hay handler. Đây là điểm thiết kế nên nhấn mạnh khi vấn đáp.

Còn phải làm gì sau khi thêm code

(1) Cài/đưa dependency của RawNet3 vào `requirements.txt`; (2) đặt checkpoint đúng đường dẫn; (3) **re-enroll** toàn bộ user (embedding cũ thuộc không gian ECAPA, không so với RawNet3 được) qua `scripts/reenroll_backend.py`; (4) **calibrate lại ngưỡng** bằng `scripts/benchmark.py` trên giọng thật.